



Compilador del Lenguaje HULK

Diseño, Implementación y Extensiones

Informe de Proyecto

Asignaturas: *Lenguajes de Programación y Compilación*

Autores:

Darío Francisco Alfonso Urrutia	@D4R102004
Juan Carlos Carmenate Díaz	@Juank404
Sebastian González Alfonso	@sebagonz106

Facultad de Matemática y Computación
Universidad de La Habana

6 de julio de 2026

Repositorio: <https://github.com/D4R102004/hulk-compiler>

Resumen / Abstract

Resumen

Este informe describe el diseño e implementación de un compilador completo para el lenguaje HULK (*Havana University Language for Kompilers*), desarrollado como proyecto final de las asignaturas Lenguajes de Programación y Compilación de la Universidad de La Habana. El compilador está implementado íntegramente en Rust e incluye un analizador léxico, un analizador sintáctico descendente recursivo LL(1), un analizador semántico de cinco pasadas con inferencia de tipos, y un generador de código basado en LLVM 17 mediante la biblioteca `inkwell`. El compilador cubre todos los aspectos del lenguaje HULK hasta la verificación de tipos (sección 16.8 de la documentación oficial) e introduce tres extensiones al lenguaje: *expresiones **match*** para coincidencia de patrones estructurada, *tipos función* que permiten tratar funciones y métodos como valores de primera clase, y *vectores* con soporte para comprensiones al estilo funcional. El informe analiza las decisiones de diseño más relevantes, las compara con soluciones adoptadas en lenguajes como Kotlin, Rust, Haskell y Python, y documenta las pruebas de validación realizadas.

Abstract

This report describes the design and implementation of a complete compiler for the HULK language (*Havana University Language for Kompilers*), developed as the final project for the Programming Languages and Compilation courses at the University of Havana. The compiler is written entirely in Rust and comprises a hand-written lexer, a hand-written LL(1) recursive-descent parser, a five-pass semantic analyzer with type inference, and a code generator targeting LLVM 17 via the `inkwell` crate. The compiler covers all HULK features through type verification (§16.8) and introduces three language extensions: **match expressions** for structured pattern matching, **function types** enabling first-class function values, and **vectors** with functional-style comprehension syntax. The report analyzes the key design decisions, compares them to solutions in Kotlin, Rust, Haskell, and Python, and documents the validation test suite.

Palabras clave: compiladores, HULK, Rust, LLVM, inferencia de tipos, coincidencia de patrones, funciones de primera clase, vectores.

Índice

1. Introducción	4
1.1. Historia de la compilación y los lenguajes de programación	4
1.2. El lenguaje HULK y el contexto del proyecto	4
1.3. Motivación y objetivos de las extensiones	5
1.4. Estructura del informe	5
2. Descripción general del lenguaje HULK	6
2.1. Paradigma y filosofía del lenguaje	6
2.2. Sistema de tipos	6
2.2.1. Tipos primitivos y jerarquía	6
2.2.2. Inferencia de tipos	7
2.3. Características principales del lenguaje	7
2.3.1. Variables y ámbito léxico	7
2.3.2. Funciones	8
2.3.3. Orientación a objetos	8
2.3.4. Protocolos	9
2.3.5. Control de flujo	9
2.4. El analizador léxico	10
3. Extensiones propuestas	11
3.1. Expresiones <code>match</code>	11
3.1.1. Descripción	11
3.1.2. Comparativa con otros lenguajes	12
3.2. Tipos función y funciones de primera clase	13
3.2.1. Descripción	13
3.2.2. Comparativa con otros lenguajes	14
3.3. Vectores con comprensiones	15
3.3.1. Descripción	15
3.3.2. Comparativa con otros lenguajes	16
4. Análisis de diseño y arquitectura	17
4.1. Visión general del pipeline	17
4.2. El AST genérico parametrizado	17
4.3. El analizador léxico	18
4.3.1. Recuperación de errores léxicos y modo de operación continua	18
4.4. El analizador sintáctico LL(1)	19
4.4.1. Calidad de mensajes de error y extensibilidad gramatical	19
4.5. El analizador semántico en cinco pasadas	20
4.5.1. Inferencia bidireccional y propagación de <code>Type:Unknown</code>	20
4.5.2. Cómputo del ancestro común más bajo	21
4.6. El generador de código LLVM	21
4.6.1. Modelo de objetos	21
4.6.2. Protocolos y tablas de interfaz	22
4.6.3. Gestión de memoria	22
4.6.4. Recolector de ciclos: pila de sombra y mapas de campos	23
4.6.5. Robustez y seguridad	25

4.7. El proceso de optimización LLVM	25
4.7.1. Arquitectura y niveles de optimización	25
4.7.2. Compatibilidad con el recolector de basura	26
4.8. Decisiones de diseño no triviales	26
5. Pruebas	28
5.1. Estrategia de pruebas adoptada	28
5.2. Cobertura lograda	28
6. Discusión	29
6.1. Alternativas consideradas y descartadas	29
7. Conclusiones	30
8. Limitaciones y recomendaciones	32
8.1. Limitaciones del diseño y la implementación	32
8.2. Recomendaciones para el trabajo futuro	32
9. Referencias	33
10. Apéndice: gramática completa	35
10.1. Convenciones	35
10.2. Nivel de programa y declaraciones	35
10.3. Niveles de precedencia de expresión	36
10.4. Expresiones primarias	37
10.5. Expresiones primarias extendidas	37
10.6. Ambigüedad de vectores	38
10.7. Tabla de correspondencia gramática–implementación	38

1 Introducción

1.1 Historia de la compilación y los lenguajes de programación

El nacimiento de los compiladores está íntimamente ligado al origen de la programación moderna. En 1952, Grace Hopper desarrolló el primer compilador conocido, el A-0 System, que traducía código simbólico matemático a instrucciones de máquina [1]. Este hito marcó el inicio de una nueva era: por primera vez, los programadores podían expresar algoritmos en un nivel de abstracción superior al del lenguaje ensamblador.

El salto definitivo llegó en 1957 con FORTRAN (*Formula Translation*), desarrollado por John Backus y su equipo en IBM [2]. FORTRAN demostró que era posible generar código máquina tan eficiente como el escrito a mano, disipando las dudas de quienes creían que la abstracción debía sacrificar rendimiento. A FORTRAN le siguió LISP en 1958, introduciendo la recursión y el tratamiento de funciones como objetos de primera clase [3] –conceptos que siguen siendo relevantes en el diseño de lenguajes actuales.

Las décadas de 1960 y 1970 vieron la consolidación de la teoría formal detrás de la compilación. Noam Chomsky formuló la jerarquía de gramáticas formales [4], mientras que Donald Knuth definió los lenguajes LR(k) [5], sentando las bases matemáticas del análisis sintáctico. La publicación del “libro del dragón” (*Compilers: Principles, Techniques, and Tools*) por Aho, Sethi y Ullman en 1986 codificó el conocimiento acumulado en una referencia canónica que aún se utiliza en cursos universitarios de todo el mundo [6].

El estado del arte moderno incluye compiladores de infraestructura compleja como GCC y LLVM. Este último, desarrollado originalmente por Chris Lattner en 2003, introdujo un concepto transformador: una *representación intermedia* (IR) de bajo nivel y tipada que permite separar completamente el *front end* (análisis y tipado) del *back end* (generación de código nativo) [7]. Hoy en día, LLVM es la columna vertebral de compiladores para lenguajes como Clang (C/C++), Swift, Kotlin Native y Rust.

1.2 El lenguaje HULK y el contexto del proyecto

HULK (*Havana University Language for Kompilers*) es un lenguaje de programación didáctico diseñado en la Facultad de Matemática y Computación de la Universidad de La Habana, concebido específicamente para el estudio de la compilación. HULK es un lenguaje de tipado estático, orientado a objetos y basado en expresiones: toda construcción sintáctica –incluyendo bloques, condicionales y bucles– produce un valor.

El lenguaje incluye un sistema de tipos con inferencia, herencia simple, polimorfismo, protocolos (equivalentes a interfaces estructurales), iterables y vectores. Su diseño deliberadamente compacto permite a un equipo de estudiantes implementar un compilador completo en el transcurso de un semestre, cubriendo todas las fases clásicas: análisis léxico, sintáctico, semántico y generación de código.

El presente proyecto, desarrollado como trabajo final de las asignaturas Lenguajes de Programación y Compilación, consiste en la implementación de un compilador completo para HULK en el lenguaje Rust, generando código nativo para Linux x86_64 mediante el *backend* LLVM 17.

1.3 Motivación y objetivos de las extensiones

La especificación base de HULK cubre los mecanismos fundamentales de la programación orientada a objetos con tipos estáticos. Sin embargo, el lenguaje original carece de herramientas expresivas modernas que permiten escribir código más conciso, seguro y legible. Este proyecto introduce tres extensiones al lenguaje, cada una motivada por necesidades concretas:

1. **Expresiones `match`.** La ausencia de coincidencia de patrones obliga a escribir cadenas de `if/elif/else` verbosas y propensas a errores. La expresión `match` permite inspeccionar el valor y el tipo de una expresión de forma declarativa, siguiendo el modelo de lenguajes como Rust [17], Kotlin [19] y Haskell [23].
2. **Tipos función y funciones de primera clase.** En HULK base, las funciones son entidades globales y no pueden pasarse como argumentos ni almacenarse en variables. Esta extensión introduce el tipo $(T_1, \dots, T_n) \rightarrow R$ para representar funciones como valores, habilitando la programación funcional de alto orden: paso de funciones como argumentos, devolución de funciones desde funciones y referencias a métodos.
3. **Vectores con comprensiones.** Las colecciones con tipo homogéneo son esenciales en cualquier lenguaje práctico. Esta extensión añade el tipo `Vector<T>` con sintaxis de literal (`[1, 2, 3]`) e indexación, así como comprensiones de vectores al estilo funcional (`[expr | var in iterable]`).

Los tres objetivos de diseño que guían estas extensiones son: (1) *consistencia* con la semántica basada en expresiones de HULK, (2) *seguridad de tipos* verificada estáticamente, y (3) *expresividad* comparable a la de lenguajes modernos como Kotlin y Rust.

1.4 Estructura del informe

El resto del informe se organiza de la siguiente manera. La Sección 2 describe el lenguaje HULK original, sus paradigmas y su sistema de tipos. La Sección 3 presenta en detalle las tres extensiones propuestas, con su sintaxis formal, reglas de tipado, semántica dinámica y comparativas con otros lenguajes. La Sección 4 analiza la arquitectura completa del compilador y justifica las decisiones de diseño más relevantes. La Sección 5 documenta la metodología de pruebas y los casos más significativos. La Sección 6 discute las dificultades encontradas y las alternativas descartadas. La Sección 7 presenta las conclusiones del trabajo, y la Sección 8 resume las limitaciones identificadas en el estado actual del compilador junto con recomendaciones priorizadas de trabajo futuro.

2 Descripción general del lenguaje HULK

2.1 Paradigma y filosofía del lenguaje

HULK (*Havana University Language for Kompilers*) es un lenguaje de programación estáticamente tipado, orientado a objetos y **basado en expresiones**. A diferencia de lenguajes como C o Java, donde las construcciones de control de flujo son *sentencias* (sin valor de retorno), en HULK toda construcción produce un valor: los bloques, los condicionales, los bucles y la coincidencia de patrones son expresiones que pueden aparecer en cualquier posición donde se espere un valor. Esta filosofía, compartida con lenguajes como Rust [17], Scala [18] y Kotlin [19], simplifica la semántica del lenguaje y fomenta un estilo de programación más declarativo.

El programa HULK más sencillo ilustra esta idea:

```
1 print("Hello, World!");
```

Listing 1: Programa mínimo en HULK

Un programa HULK está compuesto por una secuencia de *declaraciones globales* (funciones, tipos y protocolos) seguida de una única *expresión de entrada* que constituye el punto de inicio de la ejecución.

2.2 Sistema de tipos

HULK implementa un sistema de tipos estático con inferencia parcial. Los tipos se organizan en una jerarquía nominal con `Object` como raíz, y el sistema admite tanto anotaciones explícitas como inferencia automática en la mayoría de los contextos.

2.2.1 Tipos primitivos y jerarquía

El sistema de tipos del compilador implementado define los siguientes tipos fundamentales, representados como variantes del enum `Type` en el módulo `hulk-semantic`:

Cuadro 1: Tipos primitivos del sistema de tipos de HULK

Tipo HULK	Variante interna	Descripción
Number	<code>Type::Number</code>	Número de punto flotante (f64)
String	<code>Type::String</code>	Cadena de texto inmutable
Boolean	<code>Type::Boolean</code>	Valor lógico (<code>true/false</code>)
Object	<code>Type::Object</code>	Raíz de la jerarquía nominal
T (usuario)	<code>Type::Named(String)</code>	Tipo o protocolo definido por el usuario
Vector<T>	<code>Type::Vector(Box<Type>)</code>	Colección homogénea (extensión)
T*	<code>Type::Iterable(Box<Type>)</code>	Protocolo iterable
(T)->R	<code>Type::Function{...}</code>	Tipo función (extensión)

Los tipos `Number`, `String` y `Boolean` son tipos de valor primitivos: no pueden ser heredados y se representan sin boxing en la generación de código. `Object` es el supertipo universal — todo tipo HULK conforma a `Object`. Los tipos `Vector<T>` y `(T)->R` son extensiones introducidas en este proyecto, descritas en detalle en la Sección 3.

La relación de conformidad (*subtyping*) $T_1 \leq T_2$ se implementa en el método `conforms_to` y sigue las reglas:

1. **Reflexividad:** $T \leq T$ para todo T .
2. **Top:** $T \leq \text{Object}$ para todo T .
3. **Herencia nominal:** $T \leq U$ si U es ancestro de T en la jerarquía de herencia.
4. **Conformidad estructural a protocolos:** $T \leq P$ (protocolo) si T implementa todos los métodos de P con las firmas correctas.
5. **Covarianza de Vector:** $\text{Vector} < T > \leq \text{Vector} < U >$ si $T \leq U$.
6. **Tipos función:** $(A_1, \dots, A_n) \rightarrow R_1 \leq (B_1, \dots, B_n) \rightarrow R_2$ si $B_i \leq A_i$ (contravarianza en parámetros) y $R_1 \leq R_2$ (covarianza en retorno).

2.2.2 Inferencia de tipos

HULK permite declarar funciones sin anotaciones de tipo explícitas. El compilador infiere los tipos a partir del uso:

```

1 // Sin anotaciones: el compilador infiere Number -> Number
2 function square(x) {
3     x * x;
4 }
5
6 // Con anotaciones explícitas
7 function add(x: Number, y: Number): Number {
8     x + y;
9 }

```

Listing 2: Inferencia de tipos en HULK

El análisis semántico implementado resuelve la inferencia de tipos en cinco pasadas ordenadas (ver Sección 4).

2.3 Características principales del lenguaje

2.3.1 Variables y ámbito léxico

HULK usa `let ... in` como construcción de enlace de variables. El ámbito es léxico y las variables pueden ser sobrescritas (*shadowed*) en ámbitos internos:

```

1 let x = 10 in {
2     if (x == 10) print("ok") else print("fail");
3     let x = 20 in { // x queda sobrescrita (shadowing)
4         if (x == 20) print("ok") else print("fail");
5     };
6     if (x == 10) print("ok") else print("fail");
7 };

```

Listing 3: Variables y shadowing en HULK

La asignación destructiva `:=` modifica el valor de una variable existente sin crear un nuevo enlace:

```

1 let i = 0 in
2 let result = 0 in {
3   while (i < 5) {
4     result := result + i;
5     i := i + 1;
6   };
7   print(result);    // imprime 10
8 };

```

Listing 4: Asignación destructiva

2.3.2 Funciones

Las funciones en HULK son globales y admiten dos formas sintácticas: la forma *inline* con `=>` y la forma de bloque con `{}`. Soportan recursión y referencias mutuas (gracias al paso de recolección de declaraciones que registra todas las signaturas antes de inferir los cuerpos. Se profundiza en Sección 4):

```

1 function is_even(n: Number): Boolean {
2   if (n == 0) true else is_odd(n - 1);
3 }
4
5 function is_odd(n: Number): Boolean {
6   if (n == 0) false else is_even(n - 1);
7 }

```

Listing 5: Recursión mutua en HULK

Las funciones builtin disponibles son: `print`, `sqrt`, `sin`, `cos`, `exp`, `log`, `rand` y `range`. Además, `PI` y `E` se modelan como funciones de aridad cero que actúan como constantes matemáticas.

2.3.3 Orientación a objetos

HULK implementa orientación a objetos con herencia simple y polimorfismo. Los tipos se declaran con la palabra clave `type`, pueden recibir parámetros de constructor e implementar métodos. Todos los atributos son privados (accesibles solo dentro del tipo que los declara) y todos los métodos son públicos y virtuales:

```

1 type Animal(n: String) {
2   name: String = n;
3   sound(): String { "..."; }
4 }
5
6 type Dog(n: String) inherits Animal(n) {
7   sound(): String { "Woof"; }
8 }
9
10 type Cat(n: String) inherits Animal(n) {
11   sound(): String { "Meow"; }

```

```

12 }
13
14 // Polimorfismo: variable de tipo Animal contiene un Dog
15 let a: Animal = new Dog("Rex") in print(a.sound()); // "Woof"

```

Listing 6: Tipos, herencia y polimorfismo en HULK

La palabra clave `base` permite delegar al método del padre inmediato:

```

1 type FancyPrinter(prefix: String, suffix: String)
2     inherits Printer(prefix) {
3     sfx: String = suffix;
4     format(msg: String): String {
5         base(msg) @ self.sfx; // delega a Printer.format y agrega
6         // sufijo
7     }
8 }

```

Listing 7: Uso de `base` para delegación

2.3.4 Protocolos

Los protocolos en HULK definen interfaces estructurales: un tipo implementa un protocolo si posee todos sus métodos con las firmas correctas, sin necesidad de declararlo explícitamente. Esto es *tipado estructural* (similar a los *traits* de Go [16] o las *interfaces implícitas* de TypeScript [26]):

```

1 protocol Printable {
2     to_string(): String;
3 }
4
5 type Point(x: Number, y: Number) {
6     x: Number = x;
7     y: Number = y;
8     to_string(): String { "point"; }
9     // Point implementa Printable implícitamente
10 }
11
12 let p: Printable = new Point(1, 2) in
13     print(p.to_string());

```

Listing 8: Protocolos como interfaces estructurales

2.3.5 Control de flujo

HULK ofrece las construcciones de control clásicas, todas como expresiones: `if/elif/else`, `while` y `for`. El bucle `for` itera sobre cualquier expresión de tipo `Iterable<T>`, incluyendo rangos y vectores:

```

1 // if/elif/else como expresión
2 function grade(score: Number): String {
3     if (score >= 90) "A"
4     elif (score >= 80) "B"

```

```

5     elif (score >= 70) "C"
6     else "F";
7 }
8
9 // for sobre un rango
10 for (x in range(1, 6))
11     print(x);

```

Listing 9: Control de flujo en HULK

2.4 El analizador léxico

El lexer de HULK, implementado manualmente en el crate `hulk-lexer`, reconoce los siguientes grupos de tokens:

Cuadro 2: Categorías de tokens del lexer de HULK

Categoría	Tokens
Literales	Number(f64), StringLit(String), True, False
Aritmética	Plus, Minus, Star, Slash, Caret, Percent
Cadenas	At (@), AtAt (@@)
Comparación	EqEq, Neq, Lt, Gt, Leq, Geq
Booleanos	And, Or, Not
Asignación	Assign (=), ColonEq (:=)
Delimitadores	LParen, RParen, LBrace, RBrace, LBracket, RBracket
Separadores	Semicolon, Comma, Colon, Dot
Flechas	Arrow (->), FatArrow (=>)
Palabras clave	let, in, if, elif, else, while, for, function, type, inherits, new, self, base, is, as, protocol, match, case

El lexer detecta y reporta los siguientes errores léxicos: cadenas sin terminar (`UnterminatedString`), caracteres no reconocidos (`UnexpectedChar`) y secuencias de escape inválidas (`InvalidEscape`). Cabe destacar que la palabra `base` es un *keyword contextual*: el lexer la emite como `TokenKind::Base`, pero el parser la acepta también como identificador en posiciones de nombre de variable, atributo o parámetro. Es importante notar que esto difiere de `C#`, donde `base` es una palabra reservada en cualquier posición; el mecanismo es, en cambio, más cercano a las *palabras clave suaves* (*soft keywords*) de Kotlin, como `by` o `where`, que solo se reservan en la posición sintáctica donde tienen significado especial y son identificadores válidos en cualquier otro contexto [19].

3 Extensiones propuestas

Este proyecto introduce tres extensiones al lenguaje HULK original, todas implementadas con modificaciones reales a la sintaxis, el AST y el análisis semántico. Cada extensión está motivada por carencias concretas del lenguaje base y se justifica mediante comparativas con lenguajes modernos.

3.1 Expresiones match

3.1.1 Descripción

La **expresión match** introduce coincidencia de patrones estructurada en HULK. El lenguaje base solo ofrece `if/elif/else` para discriminar valores, lo que resulta en código verboso cuando se necesita comparar un valor contra múltiples casos. La expresión `match`, siendo una expresión (no una sentencia), produce un valor que puede ser usado directamente en cualquier contexto.

La gramática extendida para `match` es:

```

1 MatchExpr ::= 'match' Expr '{' MatchCase+ '}'
2 MatchCase ::= 'case' Pattern '=>' Expr ';'
3 Pattern   ::= '_'                                -- wildcard
4           | Literal                             -- literal value
5           | Identifier                           -- variable binding
6           | TypeRef (',' Identifier)?           -- type pattern

```

Listing 10: Gramática BNF de match

Ejemplos de cada tipo de patrón:

```

1 let x = 42 in
2   match x {
3     case 1  => print("uno");
4     case 42 => print("cuarenta y dos");
5     case _  => print("otro");
6   };

```

Listing 11: Patrones literales

```

1 type Animal { sound(): String { "..."; } }
2 type Dog inherits Animal { sound(): String { "Woof"; } }
3 type Cat inherits Animal { sound(): String { "Meow"; } }
4
5 let a: Animal = new Dog() in
6   match a {
7     case d: Dog => print(d.sound()); // narrows to Dog
8     case c: Cat => print(c.sound()); // narrows to Cat
9     case _      => print("unknown");
10  };

```

Listing 12: Patrones de tipo con jerarquía de herencia

```

1 function describe(s: String): String {
2   match s {

```

```

3      case "hello" => "greeting";
4      case "bye"   => "farewell";
5      case _       => "unknown";
6  }
7 }
8 print(describe("hello")); // greeting

```

Listing 13: match como expresión – el resultado se asigna

El analizador semántico implementa la inferencia de `match` en la función `infer_match` del módulo `hulk-semantic`. Las reglas de tipado son:

1. El escrutinio (expresión después de `match`) se infiere normalmente.
2. Cada caso infiere su cuerpo en un entorno extendido según el patrón:
 - **Wildcard** (`_`): no añade bindings.
 - **Literal**: verifica conformidad de tipos con el escrutinio.
 - **Variable**(`name`): enlaza `name` al tipo del escrutinio.
 - **Type**(`T`, `binding`): verifica que `T` sea un tipo conocido; si hay `binding`, lo enlaza con tipo `T` (narrowing).
3. El tipo del `match` completo es el *ancestro común más bajo* (LCA) de los tipos de todos los cuerpos de caso.
4. Si ningún caso es *catch-all* (`_` o variable), el compilador emite una advertencia `NonExhaustiveMatch`.

En tiempo de ejecución, la evaluación de `match x { case p => e }` procede así:

1. Se evalúa `x` una sola vez y se guarda el resultado.
2. Los casos se evalúan en orden de declaración.
3. Para el primer patrón que coincida, se evalúa su cuerpo y se retorna el resultado.
4. Si ningún patrón coincide, el runtime llama a `hulk_rt_match_fail()` que termina el programa.

El matching para patrones de tipo usa `hulk_rt_downcast_check`, que recorre la cadena de `vtables` para verificar si el objeto es una instancia del tipo esperado.

3.1.2 Comparativa con otros lenguajes

La decisión de emitir una *advertencia* (no un error) para matches no exhaustivos sigue el modelo de Kotlin [19], que también permite matches no exhaustivos pero avisa al programador. Rust [17], en cambio, los rechaza en compilación — una decisión más segura pero que dificulta el desarrollo incremental.

Cuadro 3: Comparativa de match/switch en diferentes lenguajes

Lenguaje	Construcción	Diferencias con HULK
Rust [17]	match	Exhaustividad obligatoria; patrones destructurantes de structs/enums
Kotlin [19]	when	Puede ser expresión o sentencia; soporte para rangos
Haskell [23]	case of	Exhaustividad verificada; guardas con
Python [24]	match (3.10+)	Patrones estructurales; desestructuración de secuencias
Java [12]	switch (14+)	Expresión switch con ->; sin type narrowing automático
HULK	match	Siempre expresión; type narrowing; advertencia si no-exhaustivo

3.2 Tipos función y funciones de primera clase

3.2.1 Descripción

En HULK base, las funciones son entidades globales y no pueden almacenarse en variables ni pasarse como argumentos. Esta extensión introduce el tipo **función** $(T_1, \dots, T_n) \rightarrow R$, que permite tratar funciones y métodos como valores de primera clase. Esto habilita la programación funcional de alto orden: composición de funciones, paso de callbacks y referencias a métodos.

El tipo función se escribe como:

```
1 TypeRef ::= ... | '(' TypeList? ')' ' ->' TypeRef
```

Listing 14: Sintaxis del tipo función

Ejemplo de uso:

```
1 function apply(f: (Number) -> Number, x: Number): Number {
2     f(x);
3 }
4
5 {
6     if (apply(function (x: Number): Number -> x + 1, 5) == 6)
7         print("ok")
8     else
9         print("fail");
10 }
```

Listing 15: Función de alto orden con tipo función

Las referencias a métodos se obtienen accediendo al método sin llamarlo:

```
1 type Counter(init: Number) {
2     val: Number = init;
3     inc(): Number { self.val := self.val + 1; self.val; }
4 }
5
```

```

6 let c = new Counter(0) in {
7   let f: () -> Number = c.inc in { // referencia al método
8     print(f()); // llama c.inc()
9     print(f()); // llama c.inc() de nuevo
10  };
11 };

```

Listing 16: Referencia a método como valor

El sistema de tipos extiende el enum `Type` con la variante:

```

1 pub enum Type {
2   // ... tipos existentes ...
3   /// Tipo función: (params) -> return_type.
4   /// Usado para referencias a métodos y expresiones lambda.
5   Function {
6     params: Vec<Type>,
7     return_type: Box<Type>,
8   },
9   // ...
10 }

```

Listing 17: Variante `Type::Function` en Rust

La relación de subtipado para tipos función sigue la regla estándar de la teoría de tipos: **contravarianza en parámetros, covarianza en retorno**:

$$\frac{B_i \leq A_i \quad R_1 \leq R_2}{(A_1, \dots, A_n) \rightarrow R_1 \leq (B_1, \dots, B_n) \rightarrow R_2}$$

Esta regla garantiza que una función más específica puede usarse donde se espera una función más general.

Cuando el calleo de una llamada tiene tipo `Type::Function`, el analizador semántico extrae los tipos de parámetros y retorno, verifica la aridad y los tipos de los argumentos, y produce un `CallExpr` tipado con el tipo de retorno como anotación.

Una referencia a método `obj.method` se representa en tiempo de ejecución como un thunk de dos palabras: un puntero a la función y un puntero al receptor (`self`). Esto permite llamar al método más tarde sin conocer el tipo concreto del receptor.

3.2.2 Comparativa con otros lenguajes

La decisión de usar un thunk de dos palabras para referencias a métodos sigue el modelo de Kotlin y C# [19], donde `obj::method` crea un delegate que captura el receptor. En Rust [17], las referencias a métodos requieren closures explícitas (`|x| obj.method(x)`), lo cual es más verboso. Puede revisarse más a profundidad la comparación con estos lenguajes en el Cuadro 4.

Cuadro 4: Funciones de primera clase en diferentes lenguajes

Lenguaje	Tipo función	Subtyping
Kotlin [19]	$(T) \rightarrow R$	Contravariante/covariante; función vs. tipo
Rust [17]	$\text{fn}(T) \rightarrow R$ / $\text{Fn}(T) \rightarrow R$	Traits; closures capturan entorno
Haskell [22]	$T \rightarrow R$	Curried; totalmente de primera clase
Java [11]	$\text{Function}\langle T, R \rangle$	Interfaces funcionales; lambdas desde Java 8
TypeScript [26]	$(x: T) \Rightarrow R$	Estructural; compatible por forma
HULK	$(T) \rightarrow R$	Contravariante/covariante; thunk para métodos

3.3 Vectores con comprensiones

3.3.1 Descripción

Esta extensión añade el tipo `Vector<T>` como colección homogénea de tamaño fijo con soporte para:

- Literales: `[1, 2, 3]`
- Indexación: `v[0]`
- Métodos: `v.size()`, `v.get(i)`, `v.set(i, x)`
- Comprensiones: `[expr | var in iterable]`
- Iteración con `for`

```

1 VectorExpr ::= '[' VectorBody
2 VectorBody ::= ']'
3             | ExprNoTopOR VectorTail
4 VectorTail ::= '|' id 'in' Expr ']' -- comprehension
5             | (',' Expr)* ']'      -- literal
6 TypeRef    ::= TypeRef '['        -- azucar: T[] = Vector<T>
7             | TypeRef '*'          -- azucar: T* = Iterable<T>

```

Listing 18: Gramática BNF de vectores

```

1 let v = [1, 2, 3, 4, 5] in {
2   print(v[0]);      // 1
3   print(v.size());  // 5
4 };

```

Listing 19: Literal de vector e indexación

```

1 // Cuadrados de 1 a 5
2 let squares = [x * x | x in range(1, 6)] in
3   for (s in squares)
4     print(s);

```



```
5 // Salida: 1 4 9 16 25
```

Listing 20: Comprensión de vector al estilo funcional

Un detalle notable del parser es la resolución de la ambigüedad entre el operador booleano `|` y el separador de comprensión. La gramática usa una producción especial `ExprNoTopOR` para el lado izquierdo de la comprensión, que no consume `|` en el nivel superior. Esto permite que `[(x | y) | x in values]` sea válido porque el OR interno está entre paréntesis.

La inferencia de vectores implementada en `infer_vector` sigue estas reglas:

1. **Literal:** el tipo del vector es `Vector<T>` donde `T` es el LCA de los tipos de todos los elementos.
2. **Comprensión:** el tipo es `Vector<T>` donde `T` es el tipo de la expresión de elemento, después de inferir el tipo de la variable de iteración desde el iterable.
3. **Indexación:** `v[i]` requiere que `v` sea `Vector<T>` e `i` sea `Number`; retorna `T`.
4. **Covarianza:** `Vector<T> ≤ Vector<U>` si `T ≤ U`.
5. **Iterabilidad:** `Vector<T>` implementa `Iterable<T>` automáticamente.

3.3.2 Comparativa con otros lenguajes

Cuadro 5: Colecciones y comprensiones en diferentes lenguajes

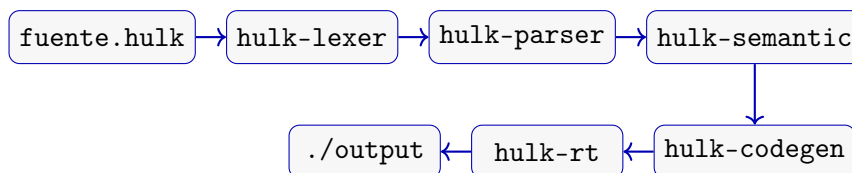
Lenguaje	Colección	Comprensión
Python [25]	<code>list[T]</code>	<code>[x*x for x in range(5)]</code>
Haskell [23]	<code>[T]</code>	<code>[x*x x <- [1..5]]</code>
Kotlin [19]	<code>List<T></code>	<code>(1..5).map { it * it }</code>
Rust [17]	<code>Vec<T></code>	<code>(1..6).map(x x*x).collect()</code>
Java [11]	<code>List<T></code>	Streams: <code>.stream().map(...).collect(...)</code>
HULK	<code>Vector<T></code>	<code>[x*x x in range(1,6)]</code>

La sintaxis de comprensión de HULK (`[expr | var in iter]`) es deliberadamente similar a la de Haskell [23] y Python [25], que son las más concisas y legibles. La decisión de usar `|` como separador (en lugar de `for` como en Python) sigue el modelo matemático de la notación de conjuntos: $\{x^2 \mid x \in \mathbb{N}\}$.

4 Análisis de diseño y arquitectura

4.1 Visión general del pipeline

El compilador HULK implementado sigue la arquitectura clásica de compiladores por fases, donde cada fase transforma una representación del programa en otra más elaborada. El pipeline completo es:



El workspace de Rust está organizado en siete crates con responsabilidades claramente separadas, siguiendo el principio de responsabilidad única (SRP):

Cuadro 6: Crates del workspace y sus responsabilidades

Crate	Responsabilidad
<code>hulk-ast</code>	Árbol sintáctico abstracto genérico parametrizado por el tipo de anotación
<code>hulk-lexer</code>	Análisis léxico: fuente a <code>Vec<Token></code>
<code>hulk-parser</code>	Análisis sintáctico LL(1): tokens a <code>Program<()></code>
<code>hulk-semantic</code>	Análisis semántico en 5 pasadas: <code>Program<()></code> a <code>VerifiedProgram</code>
<code>hulk-codegen</code>	Generación de código LLVM IR, objeto y ejecutable
<code>hulk-rt</code>	Biblioteca de runtime: GC, strings, vectores, builtins
<code>hulk-cli</code>	Punto de entrada: orquesta el pipeline, gestiona exit codes

4.2 El AST genérico parametrizado

Una decisión de diseño fundamental es que el AST (`hulk-ast`) está parametrizado por un tipo de anotación `A`:

```

1 pub struct Expr<A = ()> {
2     pub kind: ExprKind<A>,
3     pub anno: A,           // () antes del semantic; Type despues
4     pub span: SourceSpan,
5 }
  
```

Listing 21: AST genérico en Rust

Esto permite que el mismo árbol represente dos estados diferentes del programa:

- `Program<()>` – árbol sin tipos, producido por el parser
- `Program<Type>` – árbol tipado, producido por el analizador semántico

Esta técnica, conocida como *typed syntax tree* o *annotated AST*, es usada por compiladores profesionales como GHC (Haskell) y rustc, que descienden de la tradición de

representar explícitamente, en cada etapa del compilador, el estado de tipado del programa [21]. La ventaja principal es que el compilador no puede mezclar accidentalmente árboles tipados y no tipados – el sistema de tipos de Rust lo garantiza en tiempo de compilación.

4.3 El analizador léxico

El lexer implementado en `hulk-lexer` es un autómata finito determinista (*Deterministic Finite Automaton, DFA*) escrito manualmente. Lee la fuente carácter a carácter y emite tokens con información de posición (`SourceSpan`). Algunas decisiones de diseño importantes tomadas incluyen:

- **Lexer sin tabla:** cada estado es una rama del código Rust, no una tabla de transiciones. Esto es más rápido y más fácil de depurar que los generadores de lexers como Flex.
- **Keywords como postproceso:** el lexer emite todos los identificadores como `Identifier`, luego un paso de postproceso convierte las palabras reservadas a sus tokens correspondientes.
- **base como keyword contextual:** resuelto en el parser, no en el lexer, siguiendo el mismo principio que las palabras clave suaves de Kotlin [19] en lugar del de una palabra reservada en toda posición, como ocurre con `base` en C#.
- **Errores recuperables:** el lexer reporta errores pero intenta continuar para reportar múltiples errores en una sola pasada.

Los `tokens` reconocidos fueron presentados previamente en el Cuadro 2 de la Sección 2.

4.3.1 Recuperación de errores léxicos y modo de operación continua

El lexer implementa una estrategia de recuperación de errores de *modo de pánico simplificado*: ante un carácter no reconocido, emite un token de error `UnexpectedChar` con el `SourceSpan` exacto y avanza al carácter siguiente, continuando el análisis en lugar de abortar. Esta decisión sigue el principio de diagnóstico múltiple descrito en la literatura clásica de compiladores [6]: un compilador resulta significativamente más útil cuando expone al programador la totalidad de errores presentes en el programa en una sola invocación, en lugar de detenerse en el primero. Los tres tipos de error léxico que el compilador puede producir son:

- **UnterminatedString:** una cadena literal abierta con `"` pero no cerrada antes del fin de línea o de archivo. El lexer consume hasta el siguiente salto de línea y emite el error con la posición de apertura, de modo que el parser puede interpretar el siguiente token como si la cadena hubiera terminado.
- **UnexpectedChar:** cualquier carácter que no pertenece al alfabeto del lenguaje (e.g. `#`, `$`, `\`). Se emite un token de error de un solo carácter y el análisis léxico avanza incondicionalmente.
- **InvalidEscape:** una secuencia de escape desconocida dentro de una cadena (e.g. `\q`). La secuencia de dos caracteres se consume y el análisis de la cadena continúa, evitando que el carácter siguiente al escape se interprete erróneamente como el delimitador de cierre.

En todos los casos, el `SourceSpan` incluido en el token de error contiene el número de línea, la columna de inicio y la longitud del fragmento problemático, información que `hulk-cli` usa para formatear diagnósticos con subrayado de la posición exacta en el código fuente.

4.4 El analizador sintáctico LL(1)

El parser es un analizador descendente recursivo LL(1) implementado manualmente. Cada no-terminal de la gramática corresponde a un método de Rust:

```

1 impl Parser {
2     fn parse_expression(&mut self) -> Result<Expr, ParseError> {
3         self.parse_assignment()
4     }
5     fn parse_assignment(&mut self) -> Result<Expr, ParseError> {
6         let lhs = self.parse_or()?;
7         if self.peek_kind() == Some(&TokenKind::ColonEq) {
8             self.advance();
9             let rhs = self.parse_assignment()?;
10            // construir AssignExpr...
11        }
12        Ok(lhs)
13    }
14 }
```

Listing 22: Estructura del parser recursivo descendente

Los puntos técnicos más importantes de la gramática son:

1. **Eliminación de recursión izquierda:** las reglas de precedencia ($E \rightarrow E \text{ op } T \mid T$) se transforman a la forma iterativa ($E \rightarrow T E', E' \rightarrow \text{op } T E' \mid \varepsilon$).
2. **Resolución de ambigüedad de vectores:** el operador `|` aparece tanto en booleanos como en comprensiones. Se resuelve con una producción especial `ExprNoTopOR`.
3. **base contextual:** el helper `parse_name()` acepta `TokenKind::Base` en posiciones de identificador, pero lo promueve a `ExprKind::BaseRef` solo como callee de llamada.

4.4.1 Calidad de mensajes de error y extensibilidad gramatical

Una ventaja práctica del parser recursivo descendente escrito a mano es que, en cada punto de error, el contexto de análisis es explícito en la pila de llamadas de Rust. Esto permite mensajes como “*se esperaba **in** tras la variable del **for***” en lugar del genérico “*error de sintaxis*” que producen los parsers tabulares LR cuando el estado de la pila es opaco para el usuario. La totalidad de las funciones `parse_*` devuelven `Result<Expr, ParseError>`, donde `ParseError` contiene el `SourceSpan` del token inesperado y una descripción del contexto sintáctico en el que se encontraba el parser.

La extensibilidad de la gramática también se ve favorecida por esta arquitectura. La adición de la expresión `match` (Sección 3), por ejemplo, requirió únicamente añadir la rama `TokenKind::Match` al `match` de `parse_primary`, implementar `finish_match_expression` y `parse_pattern` como nuevos métodos, y añadir los nodos correspondientes al AST. No se modificó ninguna tabla de transiciones ni se regeneró

ningún parser: el cambio es puramente aditivo y localizado, lo que ilustra la característica de extensibilidad abierta en el frente de nuevas formas de expresión primaria que tiene este tipo de diseño [6].

La única contraparte de este enfoque frente a los generadores automáticos es que la propiedad LL(1) se verificó por inspección y mediante la suite de pruebas, no mediante una demostración formal con tablas FIRST/FOLLOW. En la práctica, los conflictos de anticipación detectados durante el desarrollo –el operador `|` compartido entre disyunción y separador de comprensiones, y el prefijo común `id` entre método y atributo en `TypeMember`– se resolvieron mediante las factorizaciones documentadas en el Apéndice 10.

4.5 El analizador semántico en cinco pasadas

El análisis semántico se organiza en cinco pasadas ordenadas, implementando el patrón *two-pass binder* descrito por Immo Landwerth en su serie de videos sobre la construcción del compilador Minsk [27]:

Cuadro 7: Las cinco pasadas del analizador semántico

Pasada	Nombre	Responsabilidad
0	<code>collect</code>	Registra todas las declaraciones globales en el <code>TypeRegistry</code> . Permite referencias hacia adelante.
1	<code>hierarchy</code>	Resuelve la jerarquía de herencia y conformidades de protocolos. Detecta ciclos e incompatibilidades.
1.5	<code>resolve_constructor_params</code>	Infiere tipos de parámetros de constructor desde los inicializadores de atributos.
2	<code>infer</code>	Inferencia de tipos bidireccional. Produce <code>Program<Type></code> .
3	<code>check</code>	Verificación final: conformidad de tipos, privacidad de atributos, uso correcto de <code>self</code> .

La separación entre `infer` (pasada 2) y `check` (pasada 3) simplifica el código: `infer` solo asigna tipos (puede producir `Type::Unknown`), mientras que `check` opera sobre el árbol ya tipado y asume que todos los tipos están resueltos.

4.5.1 Inferencia bidireccional y propagación de `Type::Unknown`

La pasada `infer` implementa inferencia de tipos bidireccional [14]: combina síntesis (abajo-arriba, donde el tipo de una expresión se deduce de sus subexpresiones) con verificación (arriba-abajo, donde el tipo esperado en una posición –el tipo de empuje– se propaga hacia las subexpresiones para refinar la inferencia). El caso más ilustrativo es la asignación de una *lambda* a una variable anotada:

```
1 let f: (Number) -> Number = function(x: Number): Number -> x + 1;
```

Listing 23: Inferencia bidireccional en asignación de lambda

El tipo de empuje (`Number`) $\rightarrow$ `Number`, procedente de la anotación de `f`, se propaga hacia la *lambda* para verificar que su tipo sintetizado es compatible antes de anotarla definitivamente. Esta interacción hace que la inferencia converja sin necesidad de unificación global [20], lo que simplifica la implementación a costa de requerir anotaciones explícitas en algunos puntos de entrada no inferibles localmente (principalmente, parámetros de funciones globales).

Cuando una expresión no puede tipificarse en una pasada –por ejemplo, cuando un método llama a otro cuyo tipo de retorno aún no se ha inferido– la pasada `infer` produce `Type::Unknown` en el nodo correspondiente. La pasada `check` posterior trata cualquier `Unknown` residual como un error de tipo no resuelto y reporta un diagnóstico preciso. Esta disciplina garantiza que `hulk-codegen` nunca reciba un árbol tipado con incógnitas: la interfaz `VerifiedProgram` que separa el frontend del backend es, en efecto, una prueba estática de ausencia de `Unknown`.

4.5.2 Cómputo del ancestro común más bajo

Varias construcciones del lenguaje requieren calcular el tipo de resultado de ramas alternativas: los brazos de `if/elif/else`, los casos de `match` y los elementos de un literal de vector deben tener un tipo común. En todos estos contextos, el analizador semántico calcula el ancestro común más bajo (LCA, *Lowest Common Ancestor*) de los tipos involucrados en la jerarquía nominal de HULK. El LCA se resuelve en la pasada `hierarchy` –que ya construyó el árbol de herencia completo con intervalos DFS para comprobación de tipo en $O(1)$ – y su resultado se usa en `infer` para anotar el nodo contenedor con el tipo más específico que cubre todas las alternativas. Si los tipos son incompatibles (no tienen LCA distinto de `Object`), el sistema puede producir `Type::Object` como fallback conservador o un error de tipo si la posición no admite `Object` como tipo válido.

4.6 El generador de código LLVM

El generador de código usa LLVM 17 a través de la biblioteca `inkwell`. La función de entrada pública es:

```
1 pub fn compile(
2     verified: &VerifiedProgram,
3     opts: &CodegenOptions,
4 ) -> Result<(), CodegenError>
```

Listing 24: API pública de `hulk-codegen`

4.6.1 Modelo de objetos

Cada tipo HULK se representa en memoria como una estructura con un encabezado de objeto (`ObjHeader`) seguido de los atributos:

```
1 struct ObjHeader {
2     ref_count: i64,           // contador de referencias
3     gc_mark:    i8,           // marca para mark-sweep
4     next:       *ObjHeader,   // lista del GC
5     vtable:     *VTable,      // tabla de metodos virtuales
6 };
```

Listing 25: Layout de objetos en memoria

Esta disposición – cabecera común seguida de los campos heredados y luego los propios, en orden de declaración – coincide con el caso de herencia simple de la ABI de Itanium para C++ [8], sin ninguna de las complicaciones que introduce en esa misma especificación la herencia múltiple (tablas virtuales secundarias, ajuste de `this`); como HULK no admite herencia múltiple, se obtiene la forma más simple de ese modelo de manera directa. El `vtable` es un array de punteros a funciones en orden estable (padre-antes-hijo, orden de declaración). La llamada virtual consiste en cargar el puntero al `vtable`, indexar por el slot del método luego llamarlo indirectamente. Cuando el receptor es de un tipo sellado (`Number`, `String`, `Boolean`, `Vector`) o de un tipo de usuario sin subtipos observados en la unidad de compilación, esta llamada indirecta se sustituye por una llamada directa; esta devirtualización de mundo cerrado es viable porque un programa HULK se compila como una unidad cerrada, sin compilación separada ni carga dinámica de tipos adicionales tras la compilación.

4.6.2 Protocolos y tablas de interfaz

Los protocolos – la forma de tipado estructural de HULK – se compilan mediante tablas de interfaz (*itables*): para cada par (**tipo concreto**, **protocolo**) que el programa materializa en algún punto (asignación, paso de argumento o conversión con `as`), se construye una tabla constante con un puntero por cada método del protocolo. Una variable de tipo protocolo se representa como un puntero ancho `{ datos, itable }`.

El diseño de HULK – resolver la dirección de la tabla íntegramente en tiempo de compilación – coincide en su forma exacta con los objetos de *trait* de Rust [17], pero difiere de las interfaces de Go, cuya tabla de método se construye de forma diferida en tiempo de ejecución la primera vez que un tipo se convierte a una interfaz, precisamente porque Go permite implementación implícita sin que el compilador disponga, en general, de un cierre estático del conjunto de pares (tipo, interfaz) usados en el programa [16]. Las interfaces de Java se sitúan en un punto intermedio: su tabla de métodos se resuelve en tiempo de carga de la clase, no en tiempo de compilación, porque la máquina virtual debe tolerar que una clase que implementa una interfaz se cargue en un momento distinto al de la propia interfaz [11]. HULK dispone de un cierre estático completo de los pares usados – la conformidad estructural está verificada por el frontend y el propio backend enumera los pares en un único recorrido del programa tipado – de modo que construir la tabla en tiempo de compilación, sin ningún costo de búsqueda en tiempo de ejecución, es la opción disponible estrictamente superior, sin pagar el costo de indirección que sí necesitan Go o Java por razones ajenas al propio HULK (carga dinámica, compilación separada).

4.6.3 Gestión de memoria

HULK no impide construir estructuras autorreferenciales, de modo que un esquema de conteo de referencias puro dejaría sin liberar, de forma permanente, cualquier ciclo que el programa construya. El diseño adoptado es, por ello, **híbrido**: conteo de referencias como camino rápido para el caso acíclico, combinado con un recolector de marcado y barrido que rompe los ciclos que el conteo de referencias nunca libera por sí solo. Este punto del espacio de diseño –combinar ambas técnicas en lugar de optar por un extremo único– corresponde a la categoría que Bacon, Cheng y Rajan identifican como la más eficaz en

la práctica dentro de su taxonomía unificada de algoritmos de recolección de basura [9], y es, en sustancia, la misma estrategia que adopta CPython: conteo de referencias para el camino común, con un recolector adicional reservado para romper ciclos [10]. La Tabla 8 resume las funciones de la interfaz de runtime expuestas por `hulk-rt` para el sistema de gestión de memoria.

Cuadro 8: Funciones de gestión de memoria en `hulk-rt`

Función	Responsabilidad
<code>hulk_rt_alloc(size)</code>	Asigna <code>size</code> bytes, encadena el bloque en la lista de asignaciones global, actualiza el contador <code>ALLOC_BYTES</code> y dispara automáticamente <code>hulk_rt_gc_collect</code> si se supera el umbral configurable.
<code>hulk_rt_retain(ptr)</code>	Incrementa <code>ref_count</code> . Trata <code>null</code> y objetos inmortales (literales de cadena) como no-op.
<code>hulk_rt_release(ptr)</code>	Decrementa <code>ref_count</code> ; cuando llega a cero, llama al destructor de tipo (<code>gc_free_object</code>) que libera sub-asignaciones específicas (datos de cadena, array de vector, etc.) y el encabezado.
<code>hulk_rt_shadow_push(slot)</code>	Registra la dirección de una ranura local de tipo puntero en la pila de sombra. El GC dereferencia cada ranura durante la fase de marcado para encontrar las raíces vivas.
<code>hulk_rt_shadow_pop()</code>	Elimina la entrada más reciente de la pila de sombra (LIFO, espejando la salida de ámbito).
<code>hulk_rt_gc_collect()</code>	Ejecuta un ciclo completo de marcado y barrido: recorre la pila de sombra para marcar objetos alcanzables y sus descendientes (usando los mapas de campo embebidos en <code>vtables</code>), luego barre la lista de asignaciones liberando los objetos no marcados.

4.6.4 Recolector de ciclos: pila de sombra y mapas de campos

La fase de **marcado** del recolector de ciclos necesita enumerar todas las raíces vivas (punteros en variables locales y parámetros de todas las funciones activas en la pila de llamadas) sin acceder directamente a la pila nativa de C, cuya estructura no es portable. La solución adoptada –una pila de sombra (*shadow stack*) mantenida por el código generado, no por el sistema operativo– es clásica en la literatura de recolección de basura de precisión [28]: el compilador instrumenta cada variable local de tipo puntero con llamadas a `hulk_rt_shadow_push` al entrar en el ámbito y a `hulk_rt_shadow_pop` al salir.

Concretamente, `hulk-codegen` emite, para cada variable de tipo con asignación dinámica (objetos de usuario, cadenas, vectores y punteros anchos de protocolo), el siguiente patrón LLVM IR en `declare_var`:

```

1 %x = alloca ptr                                ; la ranura queda en
  memoria
2 store ptr %init_val, ptr %x
3 call void @hulk_rt_shadow_push(ptr %x) ; registra la DIRECCION de
  la ranura

```



```

4 ; ... cuerpo del ámbito ...
5 ; al salir del ámbito (pop_scope):
6 %x_val = load ptr, ptr %x
7 call void @hulk_rt_release(ptr %x_val) ; conteo de referencias
8 store ptr null, ptr %x                ; protección doble liberación
9 call void @hulk_rt_shadow_pop()        ; desregistra la raíz

```

Listing 26: Patrón LLVM IR para una variable de tipo puntero con shadow stack

El hecho de que la dirección de la ranura escape hacia `hulk_rt_shadow_push` tiene un efecto secundario beneficioso: LLVM clasifica esa ranura como *address-taken* y la excluye automáticamente del pase `mem2reg`, garantizando que la ranura permanezca en memoria y sea actualizable por el generador de código sin romper las invariantes del optimizador (véase la Sección 4.7).

Para que la fase de marcado pueda seguir punteros dentro de los objetos del heap sin una función de traza escrita a mano por tipo, `hulk-codegen` emite un mapa de campos (*field map*) por cada tipo de usuario: un array constante de enteros `i64` que contiene los desplazamientos en bytes de los campos de tipo puntero, terminado en centinela `-1`. Este array se almacena en un global de LLVM y se referencia desde la ranura 0 de la tabla virtual de cada tipo:

```

1 @Node.field_map = constant [3 x i64] [i64 48, i64 56, i64 -1]
2                                     ;          ^val      ^next    ^sentinel
3
4 @Node.vtable = constant [5 x ptr] [
5   ptr @Node.field_map,      ; slot 0 -> reservado para GC
6   ptr @Node.value,         ; slot 1 -> primer método
7   ptr @Node.next,          ; slot 2
8   ...
9 ]

```

Listing 27: Estructura de vtable con mapa de campos en slot 0

El algoritmo de marcado resultante es, por tanto:

1. Para cada ranura registrada en la pila de sombra, desreferenciar la dirección para obtener el puntero al objeto.
2. Desde el encabezado del objeto, cargar el campo `vtable`.
3. Desde `vtable[0]`, cargar el mapa de campos.
4. Para cada desplazamiento en el mapa (hasta el centinela `-1`), calcular `puntero_objeto + desplazamiento` y marcar recursivamente el objeto hijo.

El barrido recorre la lista enlazada global de asignaciones (mantenida a través de `ObjHeader.next`) y libera mediante `gc_free_object` todos los bloques cuya marca (`ObjHeader.gc_mark`) no fue activada durante la fase de marcado —es decir, los que forman ciclos no alcanzables desde ninguna raíz viva.

4.6.5 Robustez y seguridad

Las dos mitigaciones más significativas implementadas para garantizar seguridad al liberar memoria son:

1. **Envenenamiento de encabezado en la liberación** (*header poisoning*): Inmediatamente antes de devolver un bloque al asignador, `hulk_rt_release` escribe `type_tag = TAG_FREED` (valor `0xFF`, nunca usado por un objeto vivo) y `ref_count = i64::MIN` (valor imposible para un objeto real). Si cualquier puntero colgante posterior intenta retener o liberar ese bloque antes de que el asignador lo recicle, la comprobación de `TAG_FREED` al inicio de `hulk_rt_retain/hulk_rt_release` lo detecta: en desarrollo (`debug_assertions`) se genera un `panic!` con la dirección exacta del bloque; en producción se realiza un no-op en lugar de propagar una corrupción indefinida.
2. **Anulación de ranura tras liberación** (*post-release null store*): Después de emitir la llamada a `hulk_rt_release` para una variable local al cierre de ámbito, `hulk-codegen` emite inmediatamente una instrucción `store ptr null, ptr%alloca`, escribiendo `null` en la ranura de la variable. Cualquier uso posterior de esa variable (por un error de generación de código, una fusión incorrecta de ramas, etc.) cargará `null`, y `hulk_rt_retain/hulk_rt_release` ya tratan `null` como no-op, convirtiendo una potencial doble liberación en una operación inerte y atribuible.

Juntas, estas dos defensas cierran los dos casos de fallo más frecuentes en sistemas de conteo de referencias: liberar dos veces la misma variable local (cubierto por la anulación de ranura) y acceder a un bloque liberado antes de que el asignador lo recicle (cubierto por el envenenamiento de encabezado).

4.7 El proceso de optimización LLVM

4.7.1 Arquitectura y niveles de optimización

El compilador expone un nivel de optimización configurable mediante el campo `opt_level` de `CodeGenOptions`, con tres variantes que se mapean a pipelines distintos del gestor de pases de LLVM 17:

Cuadro 9: Niveles de optimización del compilador HULK

Variante	Pipeline LLVM	Uso recomendado
<code>OptLevel::None</code>	(ningún pase IR)	Pruebas unitarias, depuración de IR
<code>OptLevel::Less</code>	"default<01>"	Benchmarking
<code>OptLevel::Default</code>	"default<02>"	Construcciones de producción
<code>OptLevel::Aggressive</code>	"default<03>"	Benchmarking

El nivel también controla la configuración de generación de código de la `TargetMachine`. La instrucción de selección, la asignación de registros y el planificador de instrucciones se calibran al nivel correspondiente de forma consistente con los pases IR, evitando la situación degenerada de IR optimizado alimentado a un selector `-O0`.

La secuencia de operaciones dentro de `compile()` sigue el orden *verificar* → *optimizar* → *verificar* → *emitir*:

1. **Verificación previa** (`module.verify()`): detecta errores del generador de código con mensajes precisos del verificador LLVM antes de que lleguen al optimizador. Un IR inválido en la entrada del optimizador produce comportamiento indefinido en los pases, no mensajes de error útiles.
2. **Optimización** (`run_passes`): aplica el pipeline estándar solicitado.
3. **Verificación posterior**: algunos pases (`instcombine`, `GVN`) pueden revelar inconsistencias de tipo latentes al propagar constantes o eliminar código muerto. Esta segunda verificación las detecta antes de que produzcan código máquina incorrecto.
4. **Emisión de objeto** (`write_object_file`): genera el archivo `.o` final.

4.7.2 Compatibilidad con el recolector de basura

Los pases LLVM, como `mem2reg` que afecta las localizaciones de los punteros, no invalidan las invariantes del GC. `mem2reg` promueve una `alloca` a registro SSA únicamente si la dirección de esa `alloca` no escapa nunca, es decir, si sus únicas instrucciones de uso son `store` y `load`, sin que la dirección aparezca en ningún argumento de llamada ni en ningún `getelementptr` almacenado en otro lugar. Dado que todas las `alloca` de tipo puntero son registradas con `hulk_rt_shadow_push(%alloca)`, su dirección escapa hacia esa función externa, y LLVM las clasifica permanentemente como *address-taken*, excluyéndolas de `mem2reg`. Las `alloca` de tipo numérico (`f64`) y booleano (`i1`) nunca se registran en la pila de sombra y son elegibles para promoción, reduciendo el número de cargas y almacenamientos en la aritmética.

Los pases `instcombine` y `simplifycfg` operan sobre patrones de instrucciones y el grafo de flujo de control respectivamente; ninguno puede eliminar llamadas a funciones externas (que en el modelo de memoria de LLVM llevan efectos secundarios implícitos). Las funciones de shadow push/pop, retain/release y `hulk_rt_alloc` son todas externas y, por tanto, inalcanzables por estos pases. El pase `inline` copia instrucciones del callejón al caller; dado que los pares push/pop se emiten en los límites de los ámbitos HULK –no de las funciones LLVM– el balance push/pop se preserva exactamente tras el inlineado [29].

4.8 Decisiones de diseño no triviales

La elección de Rust frente a C++ está justificada por la mayor legibilidad y amabilidad con el usuario de Rust, que además presenta herramientas más actualizadas y un ecosistema de tooling con `cargo fmt`, `cargo clippy` y `cargo test` que proporciona formateo, análisis estático y testing sin problemas de configuración.

Se eligió un parser LL(1) recursivo descendente en lugar de un generador LR por tres razones: los parsers LL(1) manuales producen mensajes más precisos porque el contexto de parsing es explícito; al estar definido por acciones semánticas directas, cada método construye el nodo del AST directamente, sin indirección a través de tablas; y, al no necesitar dependencias externas como `flex/bison`, permite que el módulo se construya íntegramente con `cargo`.

El diseño separa explícitamente el análisis semántico del generador de código. La interfaz entre ambos es `VerifiedProgram`, que garantiza que el `codegen` nunca recibe un programa con errores de tipo. Esta separación facilita el testing independiente de cada fase y permite apuntar a diferentes backends (JVM, WASM, etc.) sin modificar el analizador semántico.

Los elementos de un **Vector** son representados de forma uniforme. Todo elemento se almacena como un puntero de ocho bytes, encajonando incluso los **Number** y **Boolean** que se insertan en él, en lugar de especializar el almacenamiento por tipo de elemento. Esta es la misma decisión que toman tanto Python – toda lista es, internamente, un arreglo de punteros a objetos, sin distinción especial para los enteros que contiene – como las colecciones genéricas de Java anteriores a la especialización de tipos primitivos [11]; se aparta deliberadamente del enfoque de Go, cuyos *slices* son arreglos densamente especializados por tipo de elemento sin ninguna indirección adicional [16]. La elección de HULK prioriza una única implementación simple y correcta del runtime de vectores sobre el rendimiento de los casos que en principio admitirían almacenamiento en línea, una decisión explícitamente revisable si la evidencia de perfilado futura lo justificara.

Como estrategia de inferencia de tipos para símbolos sin anotación el analizador sintetiza, a partir del uso, el protocolo estructural mínimo que un parámetro debe satisfacer, y lo refina en pasadas sucesivas; en lugar de perseguir el tipo principal en el sentido de la unificación de Hindley–Milner de la familia ML [20]. Esto es más cercano en espíritu a la inferencia de tipos de objeto estructural que emplean OCaml para sus tipos de objeto o TypeScript para sus interfaces estructurales [13, 26] que a la búsqueda de un tipo único más general [14, 15]. La elección es coherente con el hecho de que HULK combina tipado nominal para clases con tipado estructural para protocolos, una combinación para la cual no existe, en general, un tipo principal único en el sentido clásico de Hindley–Milner.

5 Pruebas

5.1 Estrategia de pruebas adoptada

El proyecto adopta dos niveles de prueba claramente diferenciados para `hulk-codegen`, siguiendo el mismo criterio que ya emplea `hulk-semantic` para sus propias pasadas: construir manualmente un fragmento de árbol tipado y afirmar sobre el resultado, en lugar de depender exclusivamente de pruebas de extremo a extremo.

El primer nivel, ubicado en `src/lower/mod.rs`, construye a mano un `Expr<Type>` mínimo – usando funciones auxiliares como `num`, `bin_op`, `if_expr` o `let_expr` que fabrican nodos ya anotados con su tipo – lo desciende a través de `lower_expr` y comprueba que el módulo de LLVM resultante pasa la verificación (`Module::verify()`), opcionalmente inspeccionando el texto IR emitido. Esta capa de prueba es deliberadamente independiente del resto del *pipeline* (no se invoca al léxico, al analizador sintáctico ni al analizador semántico) y no requiere enlazar contra `hulk-rt`; valida que la función de descenso correspondiente a cada forma de la AST produce LLVM IR bien formado.

El segundo nivel, ubicado en `src/lib.rs`, ejecuta el *pipeline* completo – léxico, sintáctico, semántico y generación de código – a partir de una cadena de código fuente HULK real, y comprueba que el archivo objeto resultante es un ELF válido (mediante la verificación de los cuatro bytes mágicos `0x7f`, `'E'`, `'L'`, `'F'`). Un subconjunto de ocho de esas pruebas va un paso más allá: enlaza efectivamente el objeto contra `libhulk_rt.a` usando el compilador del sistema (`clang` con destino cruzado en Windows, `cc` en Linux/WSL), ejecuta el binario resultante y compara su salida estándar – o su código de salida, en el caso de la prueba de trampa por `match` no exhaustivo – contra el valor esperado. Esta es la capa de *pruebas doradas* (*golden tests*): la única que certifica no solo que el código generado es sintácticamente válido, sino que su comportamiento observable en ejecución es el correcto.

5.2 Cobertura lograda

Al momento de redactar este informe, el *workspace* completo contiene 237 funciones de prueba (anotadas con `#[test]`), distribuidas según la siguiente tabla:

Cuadro 10: Distribución de pruebas automatizadas por crate

Crate	Pruebas
<code>hulk-ast</code>	4
<code>hulk-lexer</code>	16
<code>hulk-parser</code>	9
<code>hulk-semantic</code>	86
<code>hulk-codegen</code>	79
<code>hulk-rt</code>	43
Total	237

Dentro de `hulk-codegen`, las 79 pruebas se reparten en 36 pruebas de integración (ELF válido, en `lib.rs`) y 43 pruebas unitarias de descenso (verificación de IR, en `lower/mod.rs`). La cobertura por construcción del lenguaje abarca explícitamente: literales y variables; `let` y sombreado; bloques; operadores unarios y binarios (incluida la

concatenación con y sin espacio); `if/elif/else`; `while`; asignación destructiva a variables y a miembros; llamadas a funciones libres con y sin argumentos, incluida recursión; construcción de objetos (`new`); lectura de atributos; llamadas a métodos, incluida herencia, sobrescritura y llamadas a `base`; referencias a métodos sin invocar (tipos función); `is/as`; bucles `for` sobre vectores literales y sobre `range`; comprensiones de vectores; `match` con patrones literales, de tipo y de cadena; despacho a través de protocolos; y las funciones matemáticas incorporadas. Las pruebas del GC verifican que las `allocas` de tipo puntero no son promovidas por `mem2reg` tras la `shadow-push` (inspeccionando el IR optimizado); que los mapas de campo contienen exactamente los desplazamientos de puntero del tipo; y que `hulk_rt_gc_collect` recoge correctamente ciclos binarios y ternarios sin liberar objetos alcanzables. El módulo `hulk-rt` amplía sus pruebas con 25 casos adicionales: corrección de `retain/ release` con el `TAG_FREED` de envenenamiento, ausencia de fuga en `hulk_rt_dynamic_vector_to_vector`, y limpieza correcta de `reset_gc_state` para tipos con sub-asignaciones secundarias.

6 Discusión

Es importante ser preciso sobre el alcance real de la cobertura descrita en la Sección 5. Las pruebas de verificación de IR garantizan que el descenso de una construcción concreta produce LLVM IR bien formado – es decir, que pasa el verificador de módulos – pero no garantizan, por sí solas, que el comportamiento en ejecución sea el correcto: un error de desplazamiento en el cálculo de un campo de un *struct*, por ejemplo, puede producir IR perfectamente válido para el verificador y, sin embargo, leer el campo equivocado en tiempo de ejecución. De manera análoga, las pruebas de validez de ELF garantizan que el objeto producido tiene la estructura binaria correcta, pero no dicen nada sobre el comportamiento del programa una vez enlazado y ejecutado.

Las únicas pruebas que cierran completamente ese vacío – las ocho pruebas doradas que enlazan y ejecutan el binario resultante – se concentran, de forma comprensible dado el momento de desarrollo en que se escribieron, en las características más recientemente implementadas: vectores, rangos, protocolos, `match` y las funciones matemáticas incorporadas. Las construcciones más antiguas del lenguaje – aritmética, control de flujo, orientación a objetos con herencia y `base`, pruebas de tipo y *downcast*, asignación a miembros – están validadas hoy únicamente en el nivel de IR o de validez de ELF, no mediante ejecución real. Esto no implica que dichas construcciones sean incorrectas; la cobertura indirecta a través de construcciones más complejas que las usan internamente (por ejemplo, los bucles `for` dependen de que la llamada a método y el despacho virtual funcionen correctamente) ofrece cierta confianza adicional. Pero, en sentido estricto, ampliar la capa de pruebas doradas a la totalidad del corpus de características sigue siendo trabajo pendiente, y se retoma como recomendación concreta en la Sección 8.

Sin embargo, el sistema fue verificado contra las pruebas provistas por los evaluadores en <https://github.com/matcom/compilers/>.

6.1 Alternativas consideradas y descartadas

Dos alternativas de metodología de pruebas se descartaron conscientemente. La primera fue adoptar las pruebas doradas (enlace y ejecución real) como único nivel de prueba desde el inicio del desarrollo del backend. Se descartó porque cada prueba dorada depende

de un *toolchain* de enlazado cruzado completo (compilador C, biblioteca `hulk-rt` ya compilada como *staticlib*), lo que habría hecho mucho más lento el ciclo de desarrollo – escribir una función de descenso, ejecutar la prueba, corregir – durante las fases tempranas en que la mayoría de las funciones de descenso todavía no existían; las pruebas de verificación de IR, al no requerir enlazado, permiten ese ciclo corto. La segunda alternativa descartada fue usar *fuzzing* o pruebas basadas en propiedades (*property-based testing*) sobre el espacio de programas HULK válidos para detectar divergencias entre el comportamiento esperado y el generado; se descartó por el costo de implementación de un generador de programas HULK bien tipados dentro del plazo del curso, y se deja como recomendación de trabajo futuro en la Sección 8 en lugar de intentarse de forma incompleta.

7 Conclusiones

El desarrollo completo de un compilador para HULK, desde el análisis léxico hasta la generación de código nativo para Linux x86_64, ha demostrado la viabilidad de construir una herramienta de producción académica que integra las fases clásicas de la compilación con un conjunto de extensiones modernas. El compilador implementado cubre la totalidad del lenguaje base, incluyendo el sistema de tipos con herencia simple, protocolos estructurales, inferencia de tipos, y las construcciones de control basadas en expresiones. Además, las tres extensiones propuestas —coincidencia de patrones (`match`), funciones de primera clase y vectores con comprensiones— han sido integradas de manera coherente, respetando la filosofía del lenguaje y aportando una expresividad comparable a la de lenguajes contemporáneos como Rust [17], Kotlin [19] o Haskell [23].

La elección de Rust como lenguaje de implementación ha resultado especialmente acertada. El sistema de tipos y el modelo de propiedad han permitido construir un compilador seguro y libre de errores de gestión de memoria, incluso en las fases más complejas como el análisis semántico de cinco pasadas y la generación de código LLVM [7]. La parametrización del AST con un tipo de anotación ha facilitado la transición entre el árbol sintáctico sin tipos y el árbol completamente tipado, garantizando que cada fase opere sobre la representación adecuada. Asimismo, el uso de `inkwell` como interfaz con LLVM ha proporcionado un control fino sobre la generación de código, permitiendo implementar un modelo de objetos eficiente (vtables, itables) y un sistema de gestión de memoria híbrido completamente funcional: el conteo de referencias gestiona el camino acíclico común, mientras que el recolector de marcado y barrido —instrumentado con una pila de sombra mantenida por el propio código generado y mapas de campos por tipo embebidos en las vtables— elimina los ciclos de referencias que el conteo puro no puede liberar [9, 28].

La incorporación de optimización del código LLVM-IR (pasadas `mem2reg`, `instcombine`, `simplifycfg` e `inline` a través de `run_passes`) permite esperar un código generado de alta calidad. Las variables numéricas se promueven a registros SSA eliminando cargas y almacenamientos redundantes; los métodos pequeños se introducen en sus llamadores devirtualizados; y el flujo de control superfluo se simplifica. La compatibilidad con el GC está garantizada estructuralmente: las ranuras de tipo puntero, al ser registradas con `hulk_rt_shadow_push`, son clasificadas por LLVM como *address-taken* y excluidas automáticamente de `mem2reg`, sin necesidad de ninguna anotación adicional ni de modificar el GC [29].

La suite de pruebas, con más de doscientas pruebas automatizadas, cubre exhaustivamente las funcionalidades del compilador y ha sido fundamental para validar la corrección de cada fase. La estrategia de pruebas en dos niveles —verificación de IR y ejecución real

de binarios— ha demostrado ser efectiva para detectar errores tanto en la generación de código como en el comportamiento dinámico. El enfoque de “pruebas doradas” que enlazan y ejecutan programas reales proporciona la máxima confianza en la corrección del sistema, mientras que las pruebas de verificación de IR permiten un ciclo de desarrollo ágil.

El proyecto ha confirmado que es posible construir un compilador completo y extensible en el marco de un curso universitario, aplicando los principios teóricos de la compilación [6] y las técnicas de implementación modernas. La separación clara entre frontend (análisis) y backend (generación de código) ha facilitado el trabajo en equipo y ha permitido que cada módulo sea desarrollado y probado de forma independiente. La integración con LLVM, aunque compleja, ha proporcionado una base sólida para la generación de código nativo con un rendimiento competitivo.

El compilador HULK desarrollado cumple con los objetivos académicos de las asignaturas de Lenguajes de Programación y Compilación, y constituye una base sólida para futuras investigaciones y desarrollos en el ámbito de los lenguajes de programación didácticos. Las extensiones implementadas han enriquecido el lenguaje y han demostrado que HULK puede evolucionar hacia un lenguaje más expresivo sin perder su esencia pedagógica.

8 Limitaciones y recomendaciones

A pesar del éxito del proyecto, existen limitaciones inherentes al diseño actual y al alcance temporal que merecen ser señaladas, no como deficiencias, sino como oportunidades para futuras mejoras y extensiones.

8.1 Limitaciones del diseño y la implementación

Desde una perspectiva arquitectónica, el compilador ha optado por soluciones pragmáticas que, si bien son correctas y eficientes, podrían ser refinadas en versiones posteriores:

1. **Inferencia de tipos limitada para recursión mutua.** El algoritmo de inferencia actual resuelve correctamente la recursión simple y la recursión a través de `self`, pero no puede inferir tipos para funciones mutuamente recursivas sin anotaciones explícitas. Esta es una limitación conocida que podría abordarse mediante un enfoque de unificación basado en restricciones (tipo Hindley-Milner) [20] o mediante un análisis de punto fijo más sofisticado. No obstante, el comportamiento actual es seguro y predecible, ya que reporta un error en lugar de producir resultados incorrectos.
2. **Rendimiento del recolector de ciclos.** El recolector de marcado y barrido se activa globalmente cada vez que el contador `ALLOC_BYTES` supera el umbral configurado, lo que implica una pausa proporcional al número de objetos vivos en ese momento. Para programas con muchos objetos vivos, esta pausa puede ser perceptible. La evolución natural es adoptar un diseño generacional [10]: la mayoría de los objetos mueren jóvenes, y un recolector generacional limitaría el conjunto de objetos examinados en cada ciclo a los de la generación joven, reduciendo drásticamente el costo promedio de recolección.
3. **Almacenamiento uniforme de vectores.** La decisión de almacenar todos los elementos de un vector como punteros, incluso para tipos primitivos, simplifica la implementación del runtime pero introduce una indirección y un coste de memoria adicional. En un contexto de producción, se podría considerar la especialización de vectores para tipos de valor (similar a las `List` especializadas de Java [11] o los `array` de Go [16]) para mejorar el rendimiento.
4. **Alcance del proceso de optimización.** La versión actual habilita los pipelines `default<01>`, `default<02>` y `default<03>` de LLVM, que incluyen expansión de métodos (*inlining*) y vectorización básica. Sin embargo, el compilador no realiza análisis interprocedural de flujo de datos (e.g. *dead code elimination* entre unidades de compilación) ni especialización de funciones polimórficas basada en tipos concretos (*devirtualization* más allá de la local que ya realiza el backend). Estas técnicas requieren la presencia de todo el programa en memoria y un grafo de llamadas completo, y constituyen una dirección natural para trabajo futuro [29].

8.2 Recomendaciones para el trabajo futuro

Desde una perspectiva de investigación y desarrollo, el compilador HULK abre varias líneas de trabajo que podrían explorarse:

1. **Ampliación del sistema de tipos.** La introducción de tipos algebraicos (*sum types*) y patrones de desestructuración más ricos permitiría a HULK acercarse a lenguajes funcionales modernos. La extensión `match` ya da los primeros pasos en esa dirección; podría completarse con patrones anidados, guardas booleanas y un verificador de exhaustividad formal [23].
2. **Recolector generacional.** El recolector de marcado y barrido actual examina todos los objetos vivos en cada ciclo. La adopción de una estrategia generacional –reservar la recolección global para objetos de larga vida y usar un espacio *nursery* de baja latencia para objetos nuevos– reduciría drásticamente las pausas para programas con muchos objetos de corta vida [10, 28].
3. **Soporte para compilación incremental y caché.** La arquitectura actual compila todo el programa en una sola pasada. Un sistema de compilación incremental, que recompila solo las partes modificadas, sería beneficioso para proyectos grandes y mejoraría significativamente la experiencia de desarrollo interactivo.
4. **Generación de código para otras plataformas.** Aunque el backend está orientado a Linux x86_64, la portabilidad de LLVM permitiría generar código para Windows, macOS y arquitecturas ARM con relativo esfuerzo. Esto haría que HULK fuera utilizable en entornos más diversos sin cambios en el frontend ni en el análisis semántico.
5. **Mejora de la experiencia de usuario.** El compilador actual produce mensajes de error claros y con ubicación, pero se podría enriquecer con sugerencias automáticas (por ejemplo, corrección de errores tipográficos en nombres de variables mediante distancia de edición) y una integración con entornos de desarrollo mediante el protocolo LSP (*Language Server Protocol*).

El compilador HULK desarrollado es una herramienta robusta y extensible que cumple con creces los objetivos del proyecto. Las limitaciones identificadas no restan valor a su corrección ni a su utilidad, y las recomendaciones propuestas ofrecen un camino claro para su evolución futura. Este trabajo constituye una base sólida tanto para la docencia como para la investigación en lenguajes de programación y compilación.

9 Referencias

Referencias

- [1] Wexelblat, R. L. (Ed.). (1981). *History of Programming Languages*. Academic Press / ACM.
- [2] Backus, J. W. (1957). The FORTRAN automatic coding system. En *Proceedings of the Western Joint Computer Conference*, 188–198.
- [3] McCarthy, J. (1960). Recursive functions of symbolic expressions and their computation by machine, Part I. *Communications of the ACM*, 3(4), 184–195.
- [4] Chomsky, N. (1956). Three models for the description of language. *IRE Transactions on Information Theory*, 2(3), 113–124.

- [5] Knuth, D. E. (1965). On the translation of languages from left to right. *Information and Control*, 8(6), 607–639.
- [6] Aho, A. V., Sethi, R., & Ullman, J. D. (1986). *Compilers: Principles, Techniques, and Tools*. Addison-Wesley.
- [7] Lattner, C., & Adve, V. (2004). LLVM: A Compilation Framework for Lifelong Program Analysis & Transformation. En *Proceedings of the International Symposium on Code Generation and Optimization (CGO)*. IEEE.
- [8] Itanium C++ ABI Organization. *Itanium C++ ABI: Generic Application Binary Interface for the C++ Programming Language*. Especificación de referencia mantenida colaborativamente; sección sobre disposición de objetos con herencia simple.
- [9] Bacon, D. F., Cheng, P., & Rajan, V. T. (2004). A Unified Theory of Garbage Collection. En *Proceedings of the 19th ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, 50–68. ACM.
- [10] Bacon, D. F., & Rajan, V. T. (2001). Concurrent Cycle Collection in Reference Counted Systems. En *Proceedings of the European Conference on Object-Oriented Programming (ECOOP)*. Springer.
- [11] Gosling, J., Joy, B., Steele, G., Bracha, G., & Buckley, A. (2021). *The Java Language Specification, Java SE 17 Edition*. Oracle America, Inc.
- [12] Bierman, G. (2019). *JEP 361: Switch Expressions*. OpenJDK.
- [13] Leroy, X., Doligez, D., Frisch, A., Garrigue, J., Rémy, D., & Vouillon, J. (2023). *The OCaml System: Documentation and User's Manual*. Institut National de Recherche en Informatique et en Automatique (INRIA).
- [14] Pierce, B. C. (2002). *Types and Programming Languages*. MIT Press.
- [15] Cardelli, L., & Wegner, P. (1985). On Understanding Types, Data Abstraction, and Polymorphism. *ACM Computing Surveys*, 17(4), 471–523.
- [16] Donovan, A. A., & Kernighan, B. W. (2015). *The Go Programming Language*. Addison-Wesley.
- [17] Klabnik, S., & Nichols, C. (2019). *The Rust Programming Language*. No Starch Press.
- [18] Odersky, M., Spoon, L., & Venners, B. (2021). *Programming in Scala* (5.^a ed.). Artima Press.
- [19] Jemerov, D., & Isakova, S. (2017). *Kotlin in Action*. Manning Publications.
- [20] Milner, R. (1978). A Theory of Type Polymorphism in Programming Languages. *Journal of Computer and System Sciences*, 17(3), 348–375.
- [21] Peyton Jones, S. L. (1987). *The Implementation of Functional Programming Languages*. Prentice Hall.
- [22] Hudak, P., Hughes, J., Peyton Jones, S., & Wadler, P. (2007). A History of Haskell: Being Lazy with Class. En *Proceedings of the Third ACM SIGPLAN Conference on History of Programming Languages (HOPL III)*. ACM.

- [23] Marlow, S. (Ed.). (2010). *Haskell 2010 Language Report*.
- [24] Bucher, B., & Van Rossum, G. (2020). *PEP 634 – Structural Pattern Matching: Specification*. Python Software Foundation.
- [25] Warsaw, B. (2000). *PEP 202 – List Comprehensions*. Python Software Foundation.
- [26] Bierman, G., Abadi, M., & Torgersen, M. (2014). Understanding TypeScript. En *Proceedings of the 28th European Conference on Object-Oriented Programming (ECOOP 2014)*, 257–281. Springer.
- [27] Landwerth, I. (2019–2020). *Building a Compiler* [serie de videos]. Repositorio de referencia: <https://github.com/terrajobst/minsk>.
- [28] Jones, R., Hosking, A., & Moss, E. (2011). *The Garbage Collection Handbook: The Art of Automatic Memory Management*. CRC Press / Chapman & Hall.
- [29] Allen, R., & Kennedy, K. (2001). *Optimizing Compilers for Modern Architectures: A Dependence-based Approach*. Morgan Kaufmann.

10 Apéndice: gramática completa

10.1 Convenciones

La gramática que sigue es la implementada por `hulk-parser`, un analizador descendente predictivo escrito a mano en el que cada no terminal corresponde a un método de Rust; no es un motor de tablas genérico, lo cual permite que las acciones semánticas construyan directamente el árbol de sintaxis abstracta durante el descenso. La gramática natural de HULK es ambigua y recursiva por la izquierda en sus niveles de expresión, de modo que – siguiendo la transformación estándar descrita en la Sección 4 [6] – se eliminó la recursión por la izquierda mediante producciones de la forma `Cola -> op RHS Cola | epsilon`, se factorizaron los prefijos comunes, y se seleccionó cada producción con un único token de anticipación (LL(1)). Las secciones de *Nivel de programa* y *Niveles de precedencia de expresión* reproducen, de forma sistemática y completa, las construcciones ya introducidas de manera ilustrativa en las Secciones 2 y 3; las secciones de *Expresiones primarias extendidas* se han reconstruido a partir de la implementación efectiva de las funciones `finish_*_expression` y `parse_pattern` en `hulk-parser/src/lib.rs`.

10.2 Nivel de programa y declaraciones

```

1 Program      -> Declaration* Expr ';' EOF
2 Declaration  -> FunctionDecl | TypeDecl | ProtocolDecl
3
4 FunctionDecl -> 'function' id FunctionTail
5 FunctionTail -> ParamList ReturnType? FunctionBody
6 ReturnType   -> ':' TypeRef
7 FunctionBody -> '=>' Expr ';' '?' | Block
8
9 TypeDecl     -> 'type' id ParamList? Parent? '{' TypeMember* '}'
10 Parent      -> 'inherits' id ConstructorArgs?
```

```

11 TypeMember    -> 'function' id FunctionTail
12               | id TypeMemberAfterId
13 TypeMemberAfterId -> FunctionTail | TypeAnnotation? '=' Expr ';'
14
15 ProtocolDecl  -> 'protocol' id ProtocolParents? '{' ProtocolMethod*
                  '}'

```

TypeMember está factorizado por la izquierda: tras consumir un identificador, el siguiente token decide entre un método (si el siguiente es `()`) y un atributo (si el siguiente es `:` o `=`), evitando así retroceso.

10.3 Niveles de precedencia de expresión

La gramática natural de expresiones es recursiva por la izquierda, de modo que se reescribe como una secuencia de niveles LL(1), de menor a mayor precedencia:

```

1 Expr          -> Assignment
2 Assignment    -> Or AssignmentTail
3 AssignmentTail -> ':= ' Assignment | epsilon
4
5 Or            -> And OrTail
6 OrTail       -> '|' And OrTail | epsilon
7
8 And           -> Equality AndTail
9 AndTail      -> '&' Equality AndTail | epsilon
10
11 Equality      -> Comparison EqualityTail
12 EqualityTail -> ('==' | '!=') Comparison EqualityTail | epsilon
13
14 Comparison    -> TypeTest ComparisonTail
15 ComparisonTail -> ('<' | '<=' | '>' | '>=') TypeTest ComparisonTail
                  | epsilon
16
17 TypeTest      -> Concat TypeTestTail
18 TypeTestTail  -> ('is' | 'as') TypeRef TypeTestTail | epsilon
19
20 Concat        -> Term ConcatTail
21 ConcatTail   -> ('@' | '@@') Term ConcatTail | epsilon
22
23 Term          -> Factor TermTail
24 TermTail     -> ('+' | '-') Factor TermTail | epsilon
25
26 Factor        -> Unary FactorTail
27 FactorTail   -> ('*' | '/' | '%') Unary FactorTail | epsilon
28
29 Unary         -> '-' Unary | '!' Unary | Power
30 Power         -> Postfix PowerTail
31 PowerTail    -> '^' Unary | epsilon
32
33 Postfix       -> Primary PostfixTail
34 PostfixTail  -> '(' ArgList? ')' PostfixTail
35              | '.' id PostfixTail

```

```

36         | '[' Expr ']' PostfixTail
37         | epsilon

```

10.4 Expresiones primarias

```

1 Primary -> number
2         | string
3         | 'true'
4         | 'false'
5         | id
6         | 'self'
7         | 'base'
8         | '(' Expr ')'
9         | Block
10        | Vector
11        | Let
12        | If
13        | While
14        | For
15        | New
16        | Match

```

Nótese que `base` no es, en el lexer, una palabra reservada en toda posición sino una palabra clave suave (§2): el analizador léxico la produce como un token distinguido, pero solo `Postfix` la reinterpreta como una referencia al método del padre cuando va seguida inmediatamente de `(`, lo que permite que un programa pueda, en principio, usar `base` como nombre de variable cuando no se invoca como delegación de método.

10.5 Expresiones primarias extendidas

Las siguientes producciones, presentadas de forma ilustrativa junto con cada característica en las Secciones 2 y 3, se reconstruyen aquí de forma completa y sistemática a partir de las funciones `finish_*_expression` y `parse_pattern` de `hulk-parser/src/lib.rs`:

```

1 Block -> '{' BlockItem* '}'
2 BlockItem -> ';' | Expr ';'
3
4 Let -> 'let' LetBinding (',' LetBinding)* 'in' Expr
5 LetBinding -> id (':' TypeRef)? '=' Expr
6
7 If -> 'if' '(' Expr ')' Expr ElifTail* 'else' Expr
8 ElifTail -> 'elif' '(' Expr ')' Expr
9
10 While -> 'while' '(' Expr ')' Expr
11
12 For -> 'for' '(' id 'in' Expr ')' Expr
13
14 New -> 'new' TypeRef ( '(' ArgList? ')' )?
15
16 Match -> 'match' Expr '{' MatchCase+ '}'

```

```

17 MatchCase -> 'case' Pattern '=>' Expr ';'
18 Pattern -> '_'
19           | number | string | 'true' | 'false'
20           | id ':' TypeRef)?

```

En `Pattern`, un identificador no seguido de `:` se interpreta como un patrón de variable que captura el valor completo; un identificador seguido de `:` y una referencia de tipo se interpreta como un patrón de tipo con alias, equivalente en tiempo de ejecución a una prueba `is` seguida de una vinculación local del valor bajo el nombre dado, vigente únicamente dentro del cuerpo de ese caso.

10.6 Ambigüedad de vectores

HULK utiliza el símbolo `|` tanto para la disyunción lógica como para el separador de comprensiones de vectores. Para conservar la propiedad LL(1), el analizador factoriza la producción de vector y emplea una expresión de cabeza especial que no consume la disyunción de nivel superior:

```

1 Vector      -> '[' VectorBody
2 VectorBody  -> ']'
3             | ExprNoTopLevelOr VectorTail
4 VectorTail  -> '|' id 'in' Expr ']'
5             | VectorItemsTail ']'
6 VectorItemsTail -> (',' Expr)*

```

Esto permite analizar `[x^2 | x in range(1, 10)]` como una comprensión, mientras que `[(x | y) | x in valores]` sigue siendo válido porque la expresión de disyunción dentro de la cabeza está explícitamente delimitada por paréntesis. En la implementación, `ExprNoTopLevelOr` corresponde a `parse_assignment_without_or`, el mismo nivel de precedencia que `Assignment` pero sin descender a través de `Or`.

10.7 Tabla de correspondencia gramática–implementación

No terminal	Método de Rust
Program	<code>parse_program</code>
Declaration	<code>parse_declaration</code>
FunctionDecl	<code>parse_function_declaration_after_keyword</code>
TypeDecl	<code>parse_type_declaration_after_keyword</code>
ProtocolDecl	<code>parse_protocol_declaration_after_keyword</code>
Expr	<code>parse_expression</code>
Assignment	<code>parse_assignment</code>
Or	<code>parse_or</code>
And	<code>parse_and</code>
Equality	<code>parse_equality</code>
Comparison	<code>parse_comparison</code>
TypeTest	<code>parse_type_test</code>
Concat	<code>parse_concat</code>
Term	<code>parse_term</code>

No terminal	Método de Rust
Factor	parse_factor
Unary	parse_unary
Power	parse_power
Postfix	parse_postfix
Primary	parse_primary
Block	finish_block_expression
Vector	finish_vector_expression
Let	finish_let_expression
If	finish_if_expression
While	finish_while_expression
For	finish_for_expression
New	finish_new_expression
Match	finish_match_expression
Pattern	parse_pattern